AI Travel Planner

Design report

26 October 2026
github.com/mugil0095/ai-travel-planner

1. Summary

I built a trip planner on LangGraph and FastAPI. A request has a destination, dates, a minimum and maximum budget, a currency, interests, a traveller count and notes. A research agent gathers information about the destination and a planner agent turns it into a daily itinerary. The graph then pauses so a person can review the draft through the API, either approving it, rejecting it with feedback, or asking for changes to named days.

There are 2 agents and 4 tools. SQLite checkpoints hold the drafts and the review history, so a restart does not lose work. A saved Lisbon session goes through all 3 review actions (section 6). The repository has 44 offline tests and 4 opt-in live checks, and I also tried crash recovery and the retry of a failed step by hand (section 7).

2. Architecture

The graph in `app/graph.py` controls the sequence from request validation to final approval. Both agents are defined in `app/agents.py`. The research agent returns a `ResearchReport` (attractions, weather, costs and sources) and the planner returns an `Itinerary` (lodging, meals and timed visits). Modification requests go back to the planner.

The FastAPI endpoints in `app/main.py` hand the work to a background runner in `app/runner.py`. The runner reads status from the SQLite checkpoints and keeps the active tasks in memory. When a draft is rejected, a feedback router decides between more research and a new plan, unless the reviewer names the target. The router is a structured model call that can use retries or fallback models.

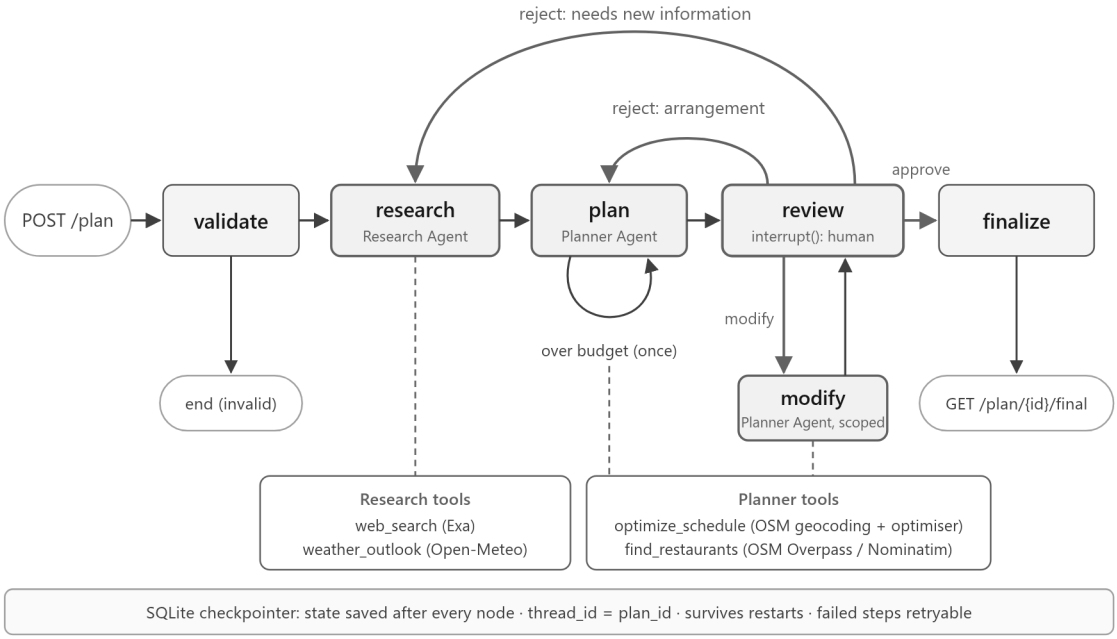


Figure 1. Research, planning and human review in the LangGraph workflow.

3. Agents and tools

Both agents use LangChain's `create_agent` loop in `app/agents.py`, with `ToolStrategy` to return Pydantic objects. Each has a model fallback chain and separate call limits for its 2 tools.

`web_search` calls Exa's `/search` endpoint with highlights enabled and asks for 5 results. Each result has a title, a URL, a publication date when present and the highlighted passages. Exa needs an API key.

`weather_outlook` uses the Open-Meteo forecast when the whole trip falls inside its 16-day window. Later trips get the observations for the same dates a year earlier, labelled as historical weather. It needs no key, and last year's weather is only a planning reference.

`optimize_schedule` geocodes places through OpenStreetMap, searching the city's bounding box first. It groups stops into days with k-means, then moves stops from full days to nearby days with room. Up to 6 stops in a day are checked in every order, and larger groups get nearest neighbour followed by 2-opt swaps.

Routes are ranked by late visits, then finish time, then travel time, and supplied opening and closing times affect the result. Travel time is the straight-line distance times 1.3, with fixed speeds and allowances for each mode, so it is an estimate. The output has the timed visits, warnings, and any places that could not be found or fitted.

`find_restaurants` sends a radius query to Overpass and sorts the OpenStreetMap results by distance. A record can include cuisine, hours, address and website. The cuisine filter is relaxed if nothing matches; the vegetarian tag filter is not. A busy response gets 1 retry per endpoint, then the tool tries a mirror and finally Nominatim, whose bounding-box search can return places outside the original radius.

4. Human review and persistence

The `review` node pauses at `interrupt()`, with `thread_id` set to the `plan_id`. A request to `POST /plan/{id}/review` resumes the saved thread. The default limit is 5 reject or modify rounds, after which the API accepts approval only.

On approval, `finalize` copies the draft unchanged into the `itinerary` field of the final response. Python adds the request details, the destination research, timestamps and the review history, with no further model call.

A rejection carries feedback, and the reviewer can set `revise` to `research` or `itinerary`. Otherwise `route_feedback()` picks the target, defaulting to the planner if the routing call fails. New research is always followed by a planning pass.

For a modification, `keep_unchanged()` restores every day the reviewer did not name, along with the lodging and the draft's top-level text, after the planner responds. If any change leaves out `day`, the code accepts the whole replacement draft, so a hotel request without a day can change anything in the plan.

The runner passes `durability="sync"`, so each checkpoint is written before the next graph step starts. A failed step can be resumed with `POST /plan/{id}/retry`. On startup the runner resumes stalled plans, while failed plans wait for a retry request and drafts awaiting review stay paused.

A plan starts as `queued`. While it runs, the status is `validating`, `researching`, `planning`, `modifying` or `finalizing`, and between checkpoint updates it can read `processing`. A paused draft is `awaiting_review`. The last four states are `completed` (the graph finished), `failed` (a step raised an error), `stalled` (the run

lost its active task) and `invalid` (the request was rejected).

Python computes the group total as lodging per night times nights, plus activities, meals and local transport per person times the number of travellers. A planning pass that comes out over `budget_max` gets 1 automatic revision. A rejection resets that allowance, and a modification goes straight back to review once the total is recalculated. Each draft is labelled `under`, `within` or `over`.

5. API

Method	Endpoint	Purpose	Responses
POST	`/plan`	Submit a travel request and start the workflow	`202` + plan_id; `422` invalid input
GET	`/plan/{id}`	Status, draft, research, reviews, progress events	`200`; `404`
POST	`/plan/{id}/review`	Approve, reject (+ feedback, optional target) or modify (+ changes)	`202`; `404`; `409` not awaiting review or limit reached; `422`
GET	`/plan/{id}/final`	The approved plan	`200`; `404`; `409` before approval
GET	`/plan/{id}/events`	Server-sent events, one per status change	`200` event stream
POST	`/plan/{id}/retry`	Resume a failed or stalled run from its checkpoint	`202`; `409` not failed

Validation rejects past or reversed dates, trips over 14 days, reversed budget bounds, empty interests and unknown modification days. The currency has to be 3 uppercase letters; I don't check it against an ISO list. The SSE stream sends a status event whenever the status, the revision or the saved event count changes.

6. Live run

The checkpoints saved on 22 October 2026 hold a Lisbon request for 20-22 November: 2 travellers, a group budget of 900-1600 EUR, and interests in food, history and viewpoints. It asked for a relaxed pace, and one traveller was vegetarian.

Research made 6 Exa searches and 1 weather call and produced 10 attractions and 9 sources. The planner then called the optimiser once and restaurant search 5 times. The first draft put Graça, Alfama, Baixa and Chiado on day 1, Belém on day 2, and Bairro Alto and Cais do Sodré on day 3. At 897 EUR for the group it was below the 900 minimum, so the budget check marked it `under`.

I rejected it and asked for less public transport, one area per day and a fado evening. The router sent the feedback to research. Fado ao Carmo, Parreirinha de Alfama and Mesa de Frades showed up in the new results, and the second draft cost 1417 EUR.

Then I modified day 2, asking for a sunset viewpoint and a vegetarian dinner nearby. The third draft put Miradouro de Santa Luzia at 16:30 and cost 1221 EUR. Days 1 and 3 and the lodging were identical to the second draft. I approved revision 3, and its contents became the `itinerary` of the final response.

Day 1 of the first draft was scheduled as follows.

Time	Activity	Travel	Cost pp
09:43-10:43	Miradouro da Senhora do Monte	13 min	free
10:53-12:53	Castelo de São Jorge	10 min	17 EUR
13:00-14:00	Miradouro de Santa Luzia	7 min	free
14:05-15:05	Sé de Lisboa	5 min	7 EUR
15:12-15:57	Arco da Rua Augusta	7 min	5 EUR
16:06-17:36	Museu Arqueológico do Carmo	9 min	7 EUR

The same draft named Pastelaria Bénard for breakfast, Casa da Tia Helena for lunch and Mezzogiorno for dinner. The times and costs above are the values saved in that draft.

Drafts 2 and 3 came from `gemini-3.5-flash-lite`, because the free-tier quota ran out during the session, and the output is weaker for it. Draft 2 scheduled fado on all 3 evenings when I had asked for one, and draft 3 kept the bookings on days 1 and 3. For the day 2 vegetarian dinner the planner searched by cuisine without setting `vegetarian=True` and chose A Muralha Tasca Tipica. Its saved restaurant record doesn't confirm a vegetarian menu.

7. Testing and live findings

There are 44 offline tests: 10 for the graph, 13 for the API, 18 for the tools, 2 for the runner and 1 for the agent wiring. The graph and API tests cover the review routes, the budget retry, persistence, HTTP responses and SSE. The tool tests cover scheduling, weather selection, restaurant fallbacks, Exa requests and budget arithmetic. The runner and wiring tests cover active-task status and agent execution inside the graph.

On 26 October 2026, `pytest -m "not live"` passed all 44 offline tests, with the 4 live ones excluded. The graph and API tests replace the agents, while the wiring test uses the real agent loop with a scripted model. The 4 opt-in checks in `test_live.py` call Open-Meteo, Nominatim, Overpass and Exa. They need `RUN_LIVE=1`, and Exa also needs a key.

The live run exposed a status race. An active plan could briefly read `invalid` between a node's writes and its checkpoint, so the runner now checks its task registry before reporting a terminal state. The 2 runner tests cover snapshots taken during an active step.

Saved errors include Gemini `503` responses and a quota response (a daily limit of 20 requests per project per model in that session). Overpass also returned `502` responses while it was overloaded. I added the model fallbacks and the restaurant retry, mirror and Nominatim sequence, though a fallback can still fail or return a worse draft.

One saved research pass made 29 web searches, after an earlier attempt kept searching until it reached the recursion limit. The middleware now caps each agent run at 8 web searches, 1 weather call, 2 scheduling calls and 10 restaurant searches. The LangGraph error record also holds an `AsyncPregelLoop` attribute error, which I traced to `checkpointer=False` on the agents combined with synchronous durability. The agents now inherit the parent checkpointer, and the wiring test covers it.

I also checked recovery by hand. I stopped the server during research, and after a restart I retried a plan that had failed on the quota error. The completed research was reused. I have not repeated those process-level checks for this revision, but `test_graph.py` covers saved-state recovery and the retry paths.

8. Design decisions and tradeoffs

Model runs can take minutes, so FastAPI returns to the client straight away and a background task keeps running the graph. The client has to poll or keep an SSE connection open. The runner's lock covers a single process, so more workers would need a shared job system and locks across processes (section 9).

Drafts and review history live in SQLite next to the graph state. Startup recovery scans the checkpoints for thread IDs, which is fine at this size but costs more as the history grows. A plan index would avoid the scan.

Pydantic checks the structure of each agent response, and plain Python computes the totals and applies the day-change guard. Place selection and price estimates are left to Gemini, so a response can pass schema validation even when a dietary requirement was never checked, as the Lisbon dinner shows.

Open-Meteo and OpenStreetMap supply data for 3 of the 4 tools without keys, while Exa and Gemini need keys. The scheduler's fixed travel speeds leave some uncertainty in arrival times.

9. What I would do next

The Lisbon run suggests two cases for an evaluation set, a request for a single fado evening and a request for a verified vegetarian meal. Trips with narrow budgets and closed attractions belong there too. Running the same requests against different prompts and Gemini models would make changes in output easier to measure.

On the tools, OSRM could give the scheduler road routes, OpenStreetMap's `opening_hours` strings need parsing, and a currency tool could attach a dated exchange rate to each conversion. Each tool would also need a defined response for missing data. For review, a small UI could show changed days beside the previous draft and accept JSON edits after schema validation. Tool-call events in SSE would show what an agent is doing between checkpoints.

Beyond a single process I would need the following.

- A queue, a shared checkpointer such as Postgres and database locks for thread resumption.
- Idempotency keys for plan submissions and review requests, since a client may repeat a request after losing its connection. Per-node timeouts, circuit breakers and caches for repeated destinations.
- Authentication and ownership checks so users keep separate plans, and per-user quotas with token and cost accounting. The 5-round limit only caps reject and modify requests.
- Logs carrying `plan_id`, traces for agent steps and metrics for latency and failures, with personal details kept out of them.
- Treating search results as untrusted input to Gemini, since pages may contain instructions aimed at the model, and checking the usage policy and capacity of the OpenStreetMap services in use.

- A retention period, scheduled cleanup and backups for the SQLite checkpoints, which hold the trip request and review history, plus restore checks for paused drafts and failed runs.

10. Assumptions

I assume the budget covers the whole group (lodging, food, activities and local transport) and excludes travel to the destination, including flights. Costs are in the request currency, and Gemini currently handles the conversion of local prices.

`TravelRequest` accepts trips of 1-14 days that start today or later, for 1-20 travellers, and nights equal days minus 1. A day-specific modification names the days to change, while a request without a day can revise the entire itinerary. Rejecting a draft requests a new research or planning pass.

Prices, opening hours and events in the itinerary come from searches and Gemini responses, so travellers need to confirm them before booking.

Sources: the application code and tests in this checkout; requirements.txt for package versions; and the saved Lisbon checkpoint history. The live API checks were not part of the 26 October test run.